

Assignment No.

05:

Testbench

Design and

Simulation using

Vivado

FAROOQ ABDULLA

Online Internship on Analog and Digital VLSI Design Flow

Batch 01

Q1. Write a Verilog testbench for a 2:1 Multiplexer module and simulate it using Vivado.

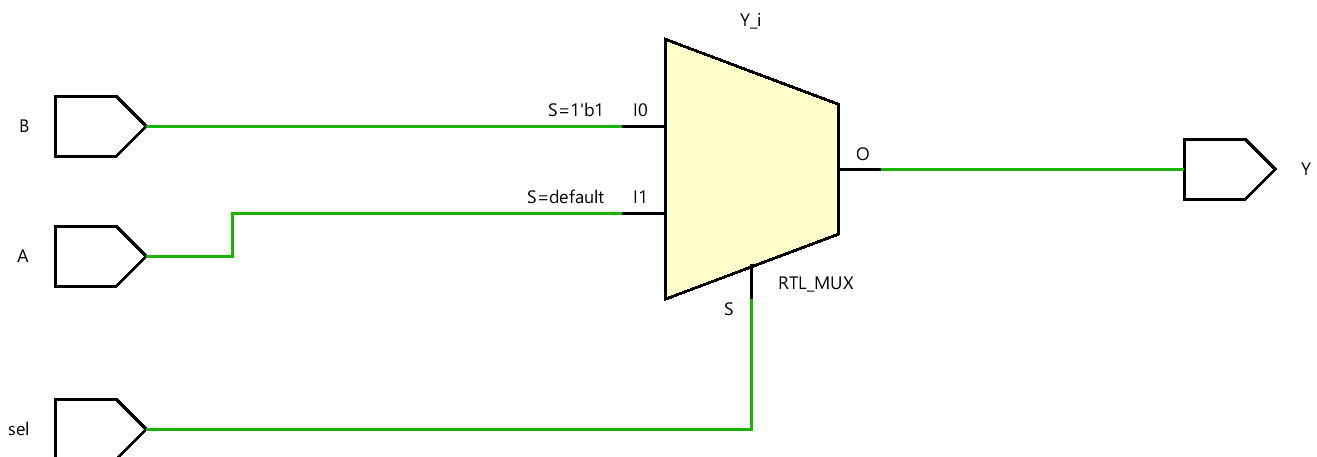
- Apply all four input combinations (sel=0,1 with A,B) using an initial block.
- Add 10 ns delay between each stimulus change and observe the output Y.
- Verify output waveform in Vivado Simulation window and capture screenshot.

DESIGN CODE:

```
`timescale 1ns/1ps

module mux2to1(
input wire A,
input wire B,
input wire sel,
output wire Y);
assign Y = sel ? B : A;
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
`timescale 1ns/1ps
module tb_mux2to1;

    reg A, B, sel;
    wire Y;

    mux2to1 DUT (.A(A),.B(B),.sel(sel),.Y(Y));

endmodule
```

```
initial begin

    $dumpfile("mux2to1.vcd");
    $dumpvars(0, tb_mux2to1);

    $display("time\tsel\tA\tB\tY");
    $monitor("%0t\t%b\t%b\t%b\t%b", $time, sel, A, B, Y);

    sel = 0; A = 0; B = 0;
    #10;

    sel = 0; A = 1; B = 0;
    #10;

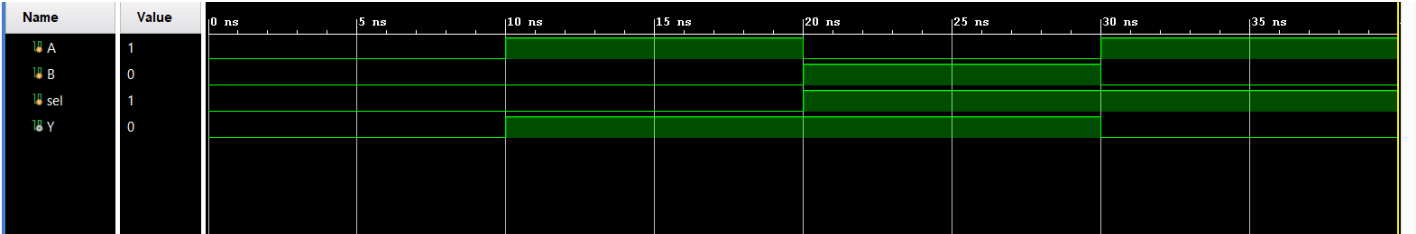
    sel = 1; A = 0; B = 1;
    #10;

    sel = 1; A = 1; B = 0;
    #10;

    $display("Simulation complete.");
    $finish;
end

endmodule
```

SIMULATION:



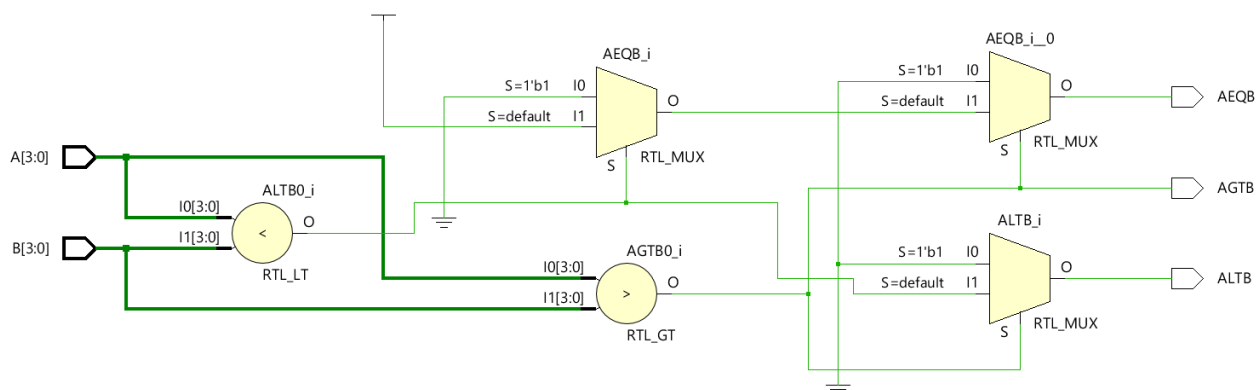
Q2. Design a self-checking testbench for a 4-bit Magnitude Comparator and simulate using Vivado.

- Generate all 4-bit input combinations for A and B using a loop.
- Use `$display` and `$monitor` system tasks to log results to the console.
- Implement automatic pass/fail checking using if-else inside the testbench.
- Verify correct outputs for $A > B$, $A = B$, and $A < B$

DESIGN CODE:

```
`timescale 1ns/1ps
module comparator4bit(
    input wire [3:0] A,
    input wire [3:0] B,
    output reg      AGTB,
    output reg      ALTB,
    output reg      AEQB);
always @(*) begin
    if (A > B) begin
        AGTB = 1'b1; ALTB = 1'b0; AEQB = 1'b0;
    end else if (A < B) begin
        AGTB = 1'b0; ALTB = 1'b1; AEQB = 1'b0;
    end else begin
        AGTB = 1'b0; ALTB = 1'b0; AEQB = 1'b1;
    end
end
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
`timescale 1ns/1ps
module tb_comparator4bit;

    reg [3:0] A, B;
    wire AGTB, ALTB, AEQB;
    integer i, j;
    integer pass_count = 0;
    integer fail_count = 0;

    reg exp_AGTB, exp_ALTB, exp_AEQB;

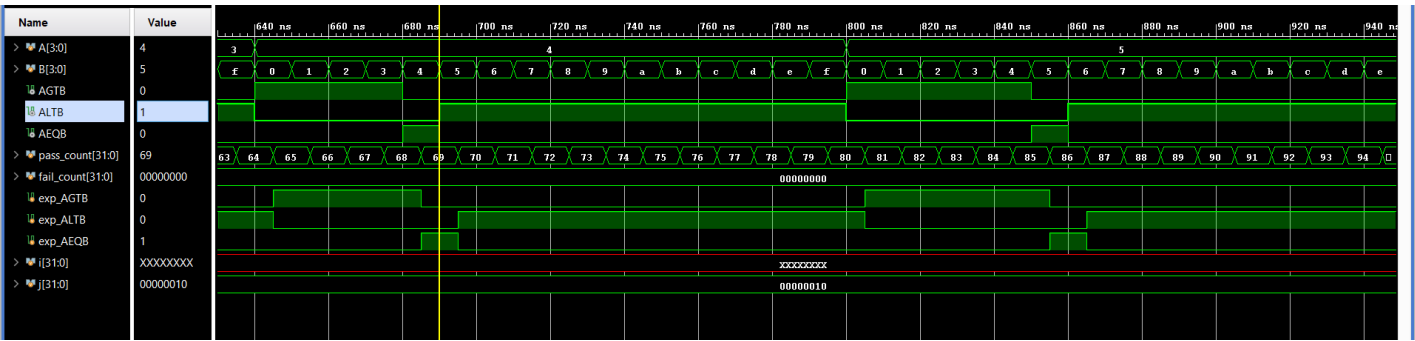
    comparator4bit DUT (.A(A), .B(B),.AGTB(AGTB), .ALTB(ALTB), .AEQB(AEQB));

    initial begin
        $monitor("t=%0t | A=%b(%0d) B=%b(%0d) | AGTB=%b ALTB=%b AEQB=%b",
            $time, A, A, B, B, AGTB, ALTB, AEQB);
    end
    initial begin
        $display("==== 4-bit Magnitude Comparator: Exhaustive Self-Check =====");

        for (i = 0; i < 16; i = i + 1) begin
            for (j = 0; j < 16; j = j + 1) begin
                A = i[3:0];
                B = j[3:0];
                #5;
                exp_AGTB = (A > B);
                exp_ALTB = (A < B);
                exp_AEQB = (A == B);
            if ((AGTB == exp_AGTB) && (ALTB == exp_ALTB) && (AEQB == exp_AEQB)) begin
                pass_count = pass_count + 1;
            end else begin
                fail_count = fail_count + 1;
            $display(">>> FAIL: A=%0d B=%0d | Got AGTB=%b ALTB=%b AEQB=%b | Expected
AGTB=%b ALTB=%b AEQB=%b",A, B, AGTB, ALTB, AEQB, exp_AGTB, exp_ALTB,exp_AEQB);
            end
            #5;
        end
    end
    $display("TOTAL TESTS = %0d | PASS = %0d | FAIL = %0d", 256, pass_count,
fail_count);
    if (fail_count == 0)
        $display(">>> RESULT: ALL TESTS PASSED <<<");
    else
        $display(">>> RESULT: SOME TESTS FAILED <<<");
    $finish;
end
```

endmodule

SIMULATION:



TCL CONSOLE:

Tcl Console Messages Log

run 1000ns
===== 4-bit Magnitude Comparator: Exhaustive Self-Check =====
t=0 | A=0000(0) B=0000(0) | AGTB=0 ALTB=0 AEQB=1
t=10000 | A=0000(0) B=0001(1) | AGTB=0 ALTB=1 AEQB=0
t=20000 | A=0000(0) B=0010(2) | AGTB=0 ALTB=1 AEQB=0
t=30000 | A=0000(0) B=0011(3) | AGTB=0 ALTB=1 AEQB=0
t=40000 | A=0000(0) B=0100(4) | AGTB=0 ALTB=1 AEQB=0
t=50000 | A=0000(0) B=0101(5) | AGTB=0 ALTB=1 AEQB=0
t=60000 | A=0000(0) B=0110(6) | AGTB=0 ALTB=1 AEQB=0
t=70000 | A=0000(0) B=0111(7) | AGTB=0 ALTB=1 AEQB=0
t=80000 | A=0000(0) B=1000(8) | AGTB=0 ALTB=1 AEQB=0
t=90000 | A=0000(0) B=1001(9) | AGTB=0 ALTB=1 AEQB=0
t=100000 | A=0000(0) B=1010(10) | AGTB=0 ALTB=1 AEQB=0
t=110000 | A=0000(0) B=1011(11) | AGTB=0 ALTB=1 AEQB=0
t=120000 | A=0000(0) B=1100(12) | AGTB=0 ALTB=1 AEQB=0
t=130000 | A=0000(0) B=1101(13) | AGTB=0 ALTB=1 AEQB=0
t=140000 | A=0000(0) B=1110(14) | AGTB=0 ALTB=1 AEQB=0
t=150000 | A=0000(0) B=1111(15) | AGTB=0 ALTB=1 AEQB=0
t=160000 | A=0001(1) B=0000(0) | AGTB=1 ALTB=0 AEQB=0
t=170000 | A=0001(1) B=0001(1) | AGTB=0 ALTB=0 AEQB=1
t=180000 | A=0001(1) B=0010(2) | AGTB=0 ALTB=1 AEQB=0
t=190000 | A=0001(1) B=0011(3) | AGTB=0 ALTB=1 AEQB=0
t=200000 | A=0001(1) B=0100(4) | AGTB=0 ALTB=1 AEQB=0
t=210000 | A=0001(1) B=0101(5) | AGTB=0 ALTB=1 AEQB=0
t=220000 | A=0001(1) B=0110(6) | AGTB=0 ALTB=1 AEQB=0
t=230000 | A=0001(1) B=0111(7) | AGTB=0 ALTB=1 AEQB=0
t=240000 | A=0001(1) B=1000(8) | AGTB=0 ALTB=1 AEQB=0
t=250000 | A=0001(1) B=1001(9) | AGTB=0 ALTB=1 AEQB=0
t=260000 | A=0001(1) B=1010(10) | AGTB=0 ALTB=1 AEQB=0
t=270000 | A=0001(1) B=1011(11) | AGTB=0 ALTB=1 AEQB=0
t=280000 | A=0001(1) B=1100(12) | AGTB=0 ALTB=1 AEQB=0
t=290000 | A=0001(1) B=1101(13) | AGTB=0 ALTB=1 AEQB=0
t=300000 | A=0001(1) B=1110(14) | AGTB=0 ALTB=1 AEQB=0

Q3. Write a complete testbench for a D Flip-Flop with synchronous reset and simulate in Vivado.

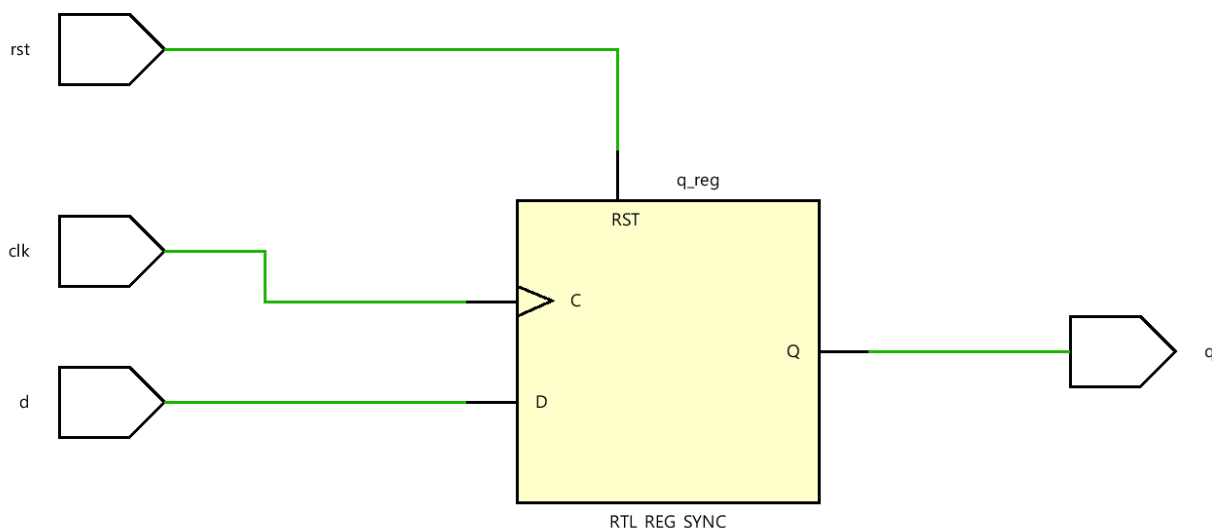
- a) Generate a clock signal with a period of 10 ns using an always block.
- b) Assert synchronous reset for the first two clock cycles, then release.
- c) Apply varied D input values and verify Q output at each rising clock edge.
- d) Use `$dumpfile` and `$dumpvars` to generate a VCD waveform file.
- e) Capture the waveform and annotate key signal transitions in the screenshot.

DESIGN CODE:

```
`timescale 1ns/1ps
module dff_sync_rst(
    input wire clk,
    input wire rst,
    input wire d,
    output reg q);

    always @(posedge clk) begin
        if (rst)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
`timescale 1ns/1ps
module tb_dff_sync_rst;

    reg clk, rst, d;
    wire q;

    dff_sync_rst DUT (.clk(clk),.rst(rst),.d(d),.q(q));

    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile ("dff_sync_rst.vcd");
        $dumpvars(0, tb_dff_sync_rst);
    end

    initial begin
        $monitor("t=%0t | clk=%b rst=%b d=%b -> q=%b", $time, clk, rst, d, q);

        rst = 1'b1;
        d = 1'b0;
        @(posedge clk);
        @(posedge clk);

        rst = 1'b0;

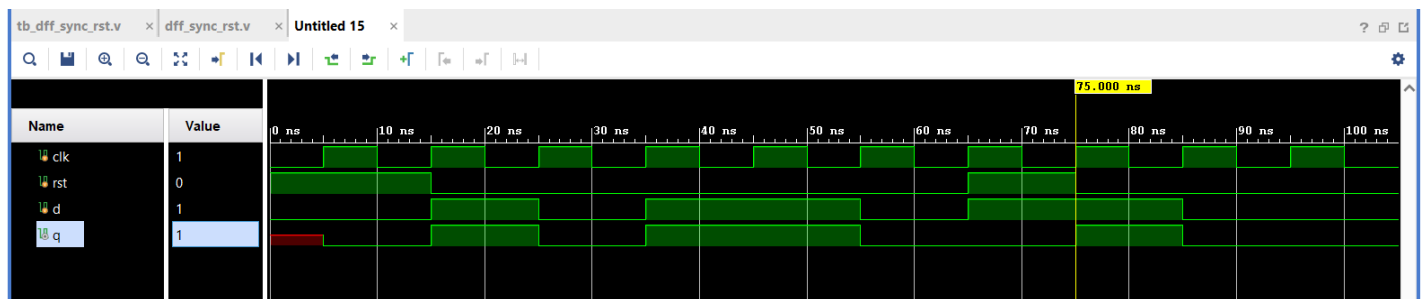
        d = 1'b1; @(posedge clk);
        d = 1'b0; @(posedge clk);
        d = 1'b1; @(posedge clk);
        d = 1'b1; @(posedge clk);
        d = 1'b0; @(posedge clk);

        rst = 1'b1; d = 1'b1; @(posedge clk);
        rst = 1'b0;

        d = 1'b1; @(posedge clk);
        d = 1'b0; @(posedge clk);

        #10;
        $display("Simulation complete.");
        $finish;
    end
endmodule
```


SIMULATION:



TCL CONSOLE:

```
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. S
#  }
#  }
# run 1000ns
t=0 | clk=0 rst=1 d=0 -> q=x
t=5000 | clk=1 rst=1 d=0 -> q=0
t=10000 | clk=0 rst=1 d=0 -> q=0
t=15000 | clk=1 rst=0 d=1 -> q=1
t=20000 | clk=0 rst=0 d=1 -> q=1
t=25000 | clk=1 rst=0 d=0 -> q=0
t=30000 | clk=0 rst=0 d=0 -> q=0
t=35000 | clk=1 rst=0 d=1 -> q=1
t=40000 | clk=0 rst=0 d=1 -> q=1
t=45000 | clk=1 rst=0 d=1 -> q=1
t=50000 | clk=0 rst=0 d=1 -> q=1
t=55000 | clk=1 rst=0 d=0 -> q=0
t=60000 | clk=0 rst=0 d=0 -> q=0
t=65000 | clk=1 rst=1 d=1 -> q=0
t=70000 | clk=0 rst=1 d=1 -> q=0
t=75000 | clk=1 rst=0 d=1 -> q=1
t=80000 | clk=0 rst=0 d=1 -> q=1
t=85000 | clk=1 rst=0 d=0 -> q=0
t=90000 | clk=0 rst=0 d=0 -> q=0
t=95000 | clk=1 rst=0 d=0 -> q=0
t=100000 | clk=0 rst=0 d=0 -> q=0
Simulation complete.
$finish called at time : 105 ns : File "C:/Users/User/NIELIT_experim
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_dff_sync_rst
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
current_sim simulation_14
close_sim
INFO: [Simtcl 6-16] Simulation closed
```

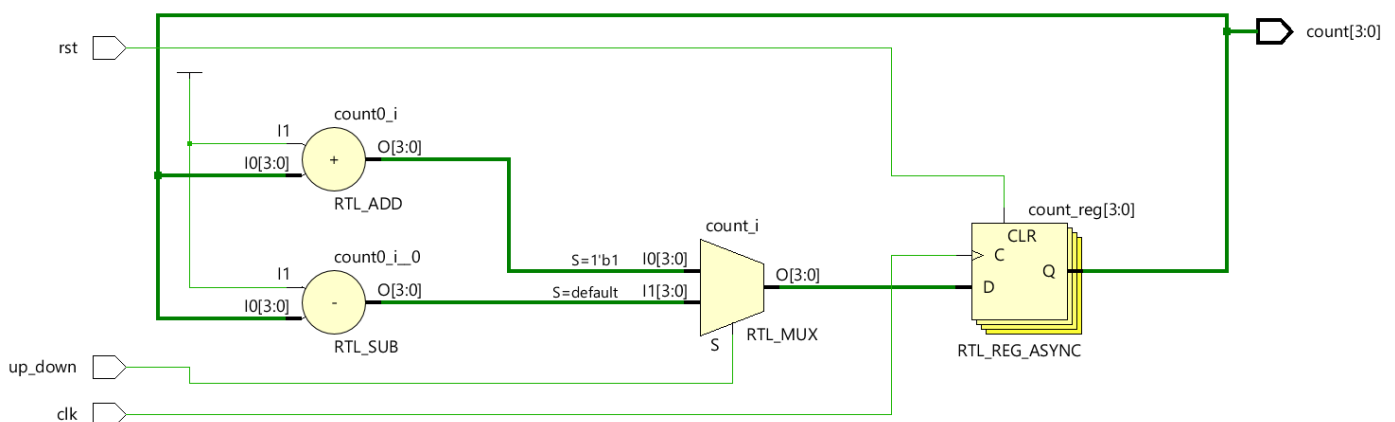
Q4. Write and simulate a testbench for the 4-bit Up/Down Counter (Assignment 04) in Vivado.

- Implement clock generation with a 10 ns time period.
- Apply asynchronous reset to initialize the counter to 0.
- Set direction control UP — verify up-counting (0 → 15) in the waveform.
- Switch direction to DOWN mid-simulation — verify down-counting (15 → 0).
- Use \$finish to terminate simulation; submit both waveform screenshots.

DESIGN CODE:

```
`timescale 1ns/1ps
module updown_counter(
    input wire    clk,
    input wire    rst,
    input wire    up_down,
    output reg [3:0] count);
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b0000;
        else if (up_down)
            count <= count + 1'b1;
        else
            count <= count - 1'b1;
    end
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
`timescale 1ns/1ps
module tb_updown_counter;

    reg          clk, rst, up_down;
    wire [3:0] count;

    updown_counter DUT (.clk(clk),.rst(rst),.up_down(up_down),.count(count));

    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("updown_counter.vcd");
        $dumpvars(0, tb_updown_counter);
    end

    initial begin
        $monitor("t=%0t | rst=%b up_down=%b -> count=%b (%0d)", $time, rst, up_down,
count, count);

        rst = 1'b1; up_down = 1'b1;
        #12;
        rst = 1'b0;

        repeat (15) @(posedge clk);
        @(negedge clk);

        $display(">>> UP counting phase complete. count = %0d (expect 15)", count);

        up_down = 1'b0;
        repeat (15) @(posedge clk);
        @(negedge clk);

        $display(">>> DOWN counting phase complete. count = %0d (expect 0)", count);

        rst = 1'b1;
        #7;
        $display(">>> Async reset check: count = %0d (expect 0)", count);
        rst = 1'b0;

        #10;
        $display("Simulation complete.");

        $finish;
    end
endmodule
```

SIMULATION:

